

Report: Robot Framework + APIs & Insights de Amazon Q

Por que “Add Random Data Generation” foi considerada, mas não implementada integralmente? [🔗](#)

A sugestão da Amazon Q para incluir uma biblioteca de geração de dados aleatórios (ex.: `data_generator.robot`) é relevante em muitos cenários, pois:

- Permite criar testes **independentes e resilientes**, reduzindo o acoplamento entre testes.
- Evita **hard-coding** de dados, promovendo manutenibilidade.
- Facilita a criação de **payloads dinâmicos** (ex.: nomes aleatórios como "João123", datas futuras para reservas, e JSONs completos com valores variados).

No entanto, **a implementação atual do projeto já contempla parte dessa estratégia:**

- A keyword `Criar Payload Para Nova Reserva` gera payloads dinâmicos para o endpoint `/booking`, aceitando argumentos personalizáveis.
- Os testes utilizam argumentos nos casos de teste para fornecer flexibilidade sem depender de dados estáticos.

A única exceção é a persistência de credenciais em `data.json`, usada intencionalmente para manter consistência e controle no ambiente de testes.

Conclusão: a necessidade de uma biblioteca dedicada de geração de dados aleatórios foi mitigada pela abordagem modular já implementada. A sugestão foi, portanto, considerada **não essencial neste estágio**.

Por que “Parallel Test Execution with Pabot” não foi viável neste projeto? [🔗](#)

Embora a execução paralela com Pabot ofereça ganhos de performance, **o design atual do projeto não suporta paralelismo de forma segura e eficiente**. Os desafios identificados foram:

Principais Limitantes: [🔗](#)

1. Sessão HTTP Compartilhada

O projeto usa um único alias de sessão (`booking_api_session`) em todos os testes. Isso causa conflitos quando múltiplos testes tentam utilizar a mesma sessão simultaneamente em processos paralelos.

2. Variáveis Globais Não Isoladas

Variáveis globais como `${GLOBAL_AUTH_TOKEN}` e `${RECURSOS_PARA_LIMPAR}` são modificadas por múltiplos processos, gerando comportamento imprevisível e falhas intermitentes.

3. Conflitos na Limpeza de Recursos

A execução paralela resulta em sobreposição nas tentativas de limpeza e atualização de recursos (como reservas e tokens), quebrando a integridade dos testes.

4. Limitação de Taxa da API (Rate Limiting)

A API Restful Booker impõe limites de requisições por período. O aumento súbito de chamadas simultâneas levou a respostas inesperadas e falhas.

5. Dependências Implícitas Entre Testes

Alguns testes dependem de estados previamente estabelecidos, como tokens compartilhados e recursos criados em etapas anteriores. Isso impede a execução independente e paralela sem uma reestruturação profunda do projeto.

Conclusão: [🔗](#)

A execução paralela foi considerada **inviável no momento atual**, e sua implementação foi postergada até que o projeto seja reestruturado para isolar sessões, variáveis e estados entre os testes.

Add Performance Testing Capabilities – Status: Implementado com Sucesso [🔗](#)

Visão Geral [🔗](#)

Os testes de carga foram integrados com sucesso ao projeto Restful Booker, utilizando o Robot Framework como base. Esta solução permite validar a performance da API de forma contínua, sem necessidade de ferramentas externas complexas.

Benefícios Estratégicos da Implementação: [🔗](#)

1. Integração Total com a Estrutura Existente

A execução de testes de carga foi incorporada diretamente ao projeto Robot Framework, permitindo que a mesma suíte seja utilizada para testes funcionais e de desempenho.

2. Baixo Overhead Operacional

A solução é leve, não requer infraestrutura extra, e pode ser facilmente integrada ao pipeline de CI/CD, permitindo execuções frequentes sem penalizar o tempo de build.

3. Detecção Antecipada de Problemas de Performance

Regressões de desempenho são identificadas no ciclo de testes, antes de impactar usuários finais ou produção.

4. Decisões Baseadas em Dados Concretos

As métricas geradas, como tempos de resposta e throughput, fornecem informações valiosas para priorizar otimizações e ajustes de infraestrutura.

5. Escalabilidade e Evolução Contínua

A estrutura é flexível e pode ser expandida para testar novos endpoints ou cenários de carga mais avançados, acompanhando o crescimento da aplicação.

Status Final: [🔗](#)

- A sugestão de **random data generation** foi considerada **redundante** dado o design atual do projeto.
- A execução paralela com Pabot foi **avaliada como inviável** sem reestruturação.
- Os testes de carga foram **implementados com sucesso**, fortalecendo a cobertura de qualidade do projeto.